Hannah D'Cruz
Dr. Ming Li
CS 3083
6 Mar 2026

Entailment Proving and Naive Bayes'

Task 1:

In the first section about Entailment Proving, I turn the knowledge base into a list of clauses, and then I negate the alpha using De Morgan's Law. Since alpha is just a bunch of ORs, the negation of it is the opposite of each variable connected by ANDs. Since each clause in the KB is assumed to be connected by ANDs, I just added the opposite of each variable in alpha into the KB.

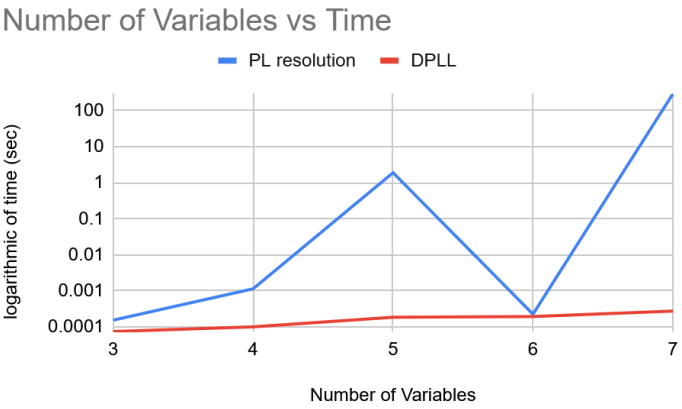
The PL resolution algorithm works by looping through each pair of clauses in the KB and applying the resolution rule to it. The resolution rule works by removing a pair of opposite variables from the input clauses, and combining the result into a new clause. It returns a list of all the newly combined clauses that are created after deleting the opposites. If the result is that both clauses become empty, then the algorithm automatically returns true, because the KB and notAlpha has an empty clause and is therefore unsatisfiable. Otherwise, it adds these new clauses into the KB and continues. If it ever loops through all of the pairs in the entire KB without making any new clauses, then it knows there is nothing more to do, and the KB and notAlpha is satisfiable, so KB does not entail alpha.

This algorithm is always complete, because it keeps making new clauses for as long as it can, so if there is an empty clause that can be made, it will find it. However, it takes a long time because it has to loop through the entire KB, usually multiple times, before it realizes that KB does not entail alpha.

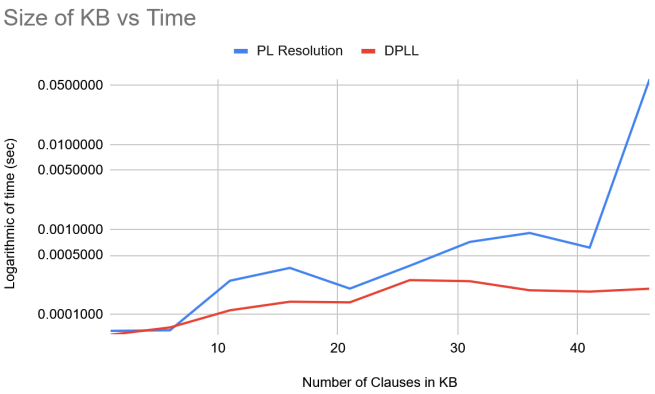
In contrast, DPLL just starts trying to assign values to the variables until it finds one that works. It starts with the easiest variables. If there are any pure variables, variables that are always negated or always not negated, you can assign one value to make all of those clauses true. Otherwise, if there are any unit clauses, clauses with only one variable in them, you know you will have to assign the value to make that clause true. If neither of those cases are true, we just pick the next variable that hasn't been assigned a value yet, try assigning both true and false to it, and pick the assignment that works. When we assign a value to a variable, we also update the KB to make those clauses true, or remove that variable from the clause because some other variable will have to make it true. We end the recursion when all of the variables have been assigned a value. At this point, either there is an empty clause in the KB because all the variables in that clause have been assigned false and removed, or all the clauses in the KB are true and the model works. Because we are trying to prove that KB and notAlpha is unsatisfiable, if we found a model that works for all the clauses, then KB does not entail alpha. If there is an empty clause in the KB, the model is unsatisfiable and KB does entail alpha.

This algorithm is always complete, because it tries all the possible combinations of assignments (except for clearly impossible ones like making a unit clause false), so it will either find a model that works, or it will learn that there is no possible model.

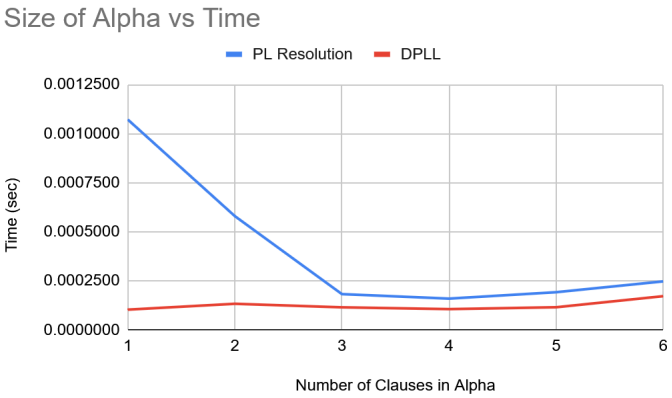
	PL Resolution	DPLL
numVars	Time (sec)	Time (sec)
3	0.0001543	0.0000743
4	0.0011439	0.0001003
5	1.8984305	0.0001850
6	0.0002257	0.0001934
7	292.9723979	0.0002740



	PL Resolution	DPLL
sizeKB	Time (sec)	Time (sec)
1	0.0000643	0.0000579
6	0.0000654	0.0000708
11	0.0002506	0.0001126
16	0.0003546	0.0001422
21	0.0002024	0.0001397
26	0.0003762	0.0002553
31	0.0007179	0.0002479
36	0.0009147	0.0001937
41	0.0006154	0.0001869
46	0.0594516	0.0002023



	PL Resolution	DPLL
sizeAlpha	Time (sec)	Time (sec)
1	0.0010722	0.0001034
2	0.0005802	0.0001332
3	0.0001830	0.0001153
4	0.0001601	0.0001063
5	0.0001927	0.0001157
6	0.0002480	0.0001720



As you can see, the DPLL algorithm was always faster. This makes sense because it ignores entire sections of the KB once it finds an assignment that makes those clauses true, while PL Resolution keeps adding to the KB, making each loop longer than the previous one. PL Resolution also took much longer to determine an entailment was false, as shown by the large spikes on the graphs, while DPLL was more consistent, since it has to assign all of the variables whether or not the KB ends up as false, while PL Resolution breaks its loop abruptly as soon as it finds that the KB is false. Since DPLL is faster overall, as well as more consistent, this makes it more scalable to larger KBs and more variables.

The most difficult part of this section for me was finding the right data types to store and process all the parts of the KB. Since the knowledge base starts as a string, I chose to turn it into a list. This allows me to have flexibility in adding and removing items easily. Each clause in the KB is a set, to prevent duplicates. Each variable is represented by a tuple holding the variable's name and a 0 or a 1 to represent whether it is true or false. This way, the clause can have both X1 and !X1, but it cannot have two X1s.

Task 2:

For Bayes' algorithm, I first had to turn each sentence from the training data into a list of words. The first part of this was tokenizing the line, which split the sentence on the spaces and removed all characters that were not letters or numbers. Then, I turned each word to lowercase. Then, I could check if any of the words were stopwords according to nltk, and remove the words in my sentence that matched the list of stopwords. Once I had the shorter list of words, I could use the lemmatizer to get the standard form of each word (fixing plural nouns, verb tenses, etc). I chose to lemmatize after I removed stopwords because the stopwords list has all the versions of the stopwords, so it wouldn't miss any even if they aren't lemmatized. Additionally, the lemmatizer is relatively slow, so the fewer times I have to run it, the better.

Once I had the cleaned list of words from the sentence, I could update the global vocabulary list with the new occurrences. The vocabulary is a dictionary with each word as the key and a list of its occurrence in each subject as the value. I incremented the appropriate subject index for each word in the sentence, or made a new entry in the dictionary if necessary. The next step in building the dictionary is Laplacian smoothing. We increment each count by one. This approximates the words' relative frequencies very well, while accounting for the fact that words might show up in a subject even if they were not in the training data, so the probability of each subject should never be 0. The final step in building the vocabulary is to divide each count by the total number of sentences for that subject, so that the values become relative frequencies.

To apply this model to the testing data, we first need to split the testing sentence into relevant words, using the same method we used for the training data. It is important to use the same method so that there are more likely to be matches found between the testing and the training data. Then, for each subject, we multiply the probabilities of each word together to get the probability that the whole sentence belongs to that subject. It is also important to multiply by the proportion of training data sentences from that subject, because subjects with fewer training sentences would have lower frequencies, which should not affect the final probabilities. The algorithm labels the sentence with the subject with the highest probability.

The second dataset I chose was a sentiment analysis of online reviews. This algorithm worked very similarly to the subject dataset, except since positive and negative reviews were mixed together, when building the vocabulary I had to check whether each

sentence was positive or negative to know whether I should increment the positive or the negative index in the vocab.

Subject classification:

```
answer: b e h p
guesses:43 4 1 3
guesses:0 49 1 0
guesses:0 1 49 0
guesses:2 2 1 44
```

sentiment classification:

```
answer: + -
guesses:32 2
guesses:9 32
```

Overall, my vocabularies were both very accurate, as shown in the charts. However, it did confuse a few sentences. The most common confusion for the subject analysis was guessing economics about a biology sentence. For example, it assigned “The human genome project was a major milestone in modern science." as economics. Out of the words, human was equally probable to be biology or economics, genome did not occur frequently enough in biology compared to the others to make a big difference, project occurred most often in economics, major occurred in all subjects about equally, milestone was not in the vocabulary, modern occurred slightly more often in history, and science was not in the vocabulary. This led to economics being just slightly most likely. The most common confusion in the sentiment analysis was guessing negative when it should have been positive. For example, it assigned "In other words, the content level of this film is enough to easily fill a dozen other films." as a negative sentence when it should have been positive. This is difficult to tell, even for a human. The algorithm thought word, level, film, easily, and dozen were about equally likely in either positive or negative sentences. It incorrectly identified “enough” and “fill” as negative words; they are likely usually negated as not enough or does not fill.